XDA003-H Hook 操作說明

XDA003-H 操作邏輯說明

XDA003-H（掛鉤端） 是 XDA003 工安穿戴系統的鉤體裝置，與 XDA003-B（腰帶端） 搭配使用。Hook 端透過磁場感測判斷自己是否「確實掛在合格金屬結構上」，並以 BLE peripheral 角色持續廣播 / 推送狀態給 Belt。本文件以使用者視角描述 Hook 在實際工作中的行為。

> Hook 端是 always-on 架構（Phase 1），平時不進 deep sleep；只有在「沒有 BLE client 連著」與「充電中」兩個情境下才會用 deep sleep duty-cycle 省電。詳見 §3 / §4。

---

1. 系統啟動與初始化

* 開機後自動運作，無需手動操作。

* Hook 在 setup() 階段一次性初始化下列資源並進入持續運作的 loop：

* LittleFS 載入既有的設定（警報窗口、X/Y/Z 三軸靈敏度、電池 ADC 校正值），找不到就寫入韌體預設值。

* 磁場感測器（QMC5883L）I²C 初始化。

* UART AT 命令服務（用於 USB/Serial 工具下發 `AT+BATT_FACTOR` / `AT+BATT_OFFSET` / `AT+RESET_SETTING=1`）。

* BLE peripheral：建立 GATT 服務（含 OTA + Device Information Service），開始廣播。

* 裝置名規則：BLE 裝置名 = `HOOK-` + ESP32 efuse MAC 前 3 byte（例如 `HOOK-C82CF8`）。同一批量產的 OUI 末三 byte 可能相同，前三 byte 通常足以唯一識別，不需要量產時逐台寫入名字。

* Hook 序號（6 byte）：開機時從 efuse MAC 取出來放進廣播 manufacturer data，這是 Belt 端用來辨識「這支是不是綁定的掛鉤」的唯一識別碼。

* 開機檢查充電器：setup() 進入正式運作前會先讀 `pCHARGING` 腳位；若為 HIGH 表示正在充電，直接進入充電模式（見 §4），不會啟動磁場感測與警報邏輯。

* 首次安裝 / 出廠未燒韌體：第一次需透過 USB 燒錄韌體；之後即可走 BLE OTA（見 §5）。

---

2. 主要功能與邏輯流程

2.1 BLE 廣播與連線

* Hook 是 BLE peripheral，永遠主動廣播讓 Belt 連線。廣播包拆成兩段以避開 31-byte 限制：

* Primary ADV：Flags + Manufacturer Data（Hook 序號、status、未偵測累積秒數、電量、電源模式 — 共 10 bytes）。被動掃描器不需連線就能讀到當前狀態。

* Scan Response：128-bit Service UUID + 裝置名（`HOOK-XXXXXX`）。主動掃描器才看得到。

* GATT Service UUID `83940000-5273-9374-2109-847320948571`，內含兩支 characteristic：

* TX (notify)：Belt 訂閱後，Hook 主動推送磁場數值、狀態、ACK / NACK。

* RX (write)：Belt 寫入 17 byte 設定包，Hook 立即驗證、套用、回傳 ACK。

* 連線管理：

* 沒有 client 連線時持續廣播。

* 有 client 連線後保持連線、推送 notify。

* client 斷線會在 `onDisconnect` callback 立即重啟廣播。

* Hook 接受任何符合 GATT 規格的 client 連線（不限定 Belt 的 MAC）— 所以開發 / 維護時可以直接用 Web BLE Console（XDA003 Belt repo 的 tools/）以瀏覽器連上同一支 Hook，與 Belt 共用同一條 GATT 通道做即時監控。

2.2 磁場感測與「掛接狀態」判定

* 背景 task 每 500 ms 取一次 QMC5883L 三軸（X / Y / Z）原始讀值，並依據設定中每個軸的條件判斷是否「偵測到金屬」：

* 每個軸獨立設定 `sensing_type`（INCLUSIVE / EXCLUSIVE）+ `sensitivity_high` + `sensitivity_low`。

* INCLUSIVE：軸絕對值落在 \[low, high\] 內 → 該軸視為偵測到金屬。

* EXCLUSIVE：軸絕對值落在 \[low, high\] 外 → 該軸視為偵測到金屬。

* 三軸 OR：任何一軸命中即視為「本次偵測到金屬」。

* 每次量測都會即時更新：

* `last_metal_seen_ms`：上次偵測到金屬的時刻（毫秒）。

* `not_detected_counter`：上次正常掛接到現在已連續多少秒沒看到金屬（0~255 上限）。後台用這個欄位分析「作業員多久沒掛回」。

* `status`：0 = 正常掛接；1 = 警報。

* 整段判斷是時間窗口式（基於 `millis()`），不是計數器式 — 不論主迴圈或 mag task 跑多快，警報觸發時間都以「實際秒數」為準。

2.3 警報邏輯（Alarm）

* 觸發條件：連續 `system_cycle_time` 秒內完全沒偵測到金屬就進入警報狀態（`status = 1`）。

* `system_cycle_time` 預設 10秒，可由 Belt（或 Web BLE Console）透過 17 byte 設定包寫入 1~30秒之間任何值。

* 寫入後立即生效，並透過 LittleFS 持久保存，重開機後仍沿用。

* 警報行為：

* 兩顆 LED（`pLed`、`pLed1`）同步亮起，提醒作業員「這支 Hook 沒有掛在合格金屬上」。

* Hook 同時在 §2.4 的 BLE notify 把 `status = 1` 推給 Belt；Belt 端再依「兩支 Hook 同時異常 + 位於危險區」決定是否升級為遠端緊急上報（見 [21_XDA003B Belt 操作說明](../belt/operation.md) §2.4）。

* 解除條件：再次偵測到金屬即立即清零 — `last_metal_seen_ms` 更新到當下，`not_detected_counter` 歸 0，`status` 切回 0，LED 熄滅。中途短暫掛上又拿開不會被「累計」到下次警報。

* Hook 端不負責判斷「危險區 / 安全區」與「兩支 Hook 同時異常」這類聚合邏輯，那是 Belt 的職責。Hook 只忠實回報自己的物理狀態。

2.4 BLE Notify 推送（連線中）

當有 BLE client 連線時，Hook 會主動推送下列 notify 訊息（首 byte 為類型）：

Type	內容	觸發
`0x00`	+ 17B 實際儲存的設定值	client 寫入長度正確的設定 → ACK，client 可比對是否被韌體 clamp（例如 `system_cycle_time` 超出 1~30 範圍會被改成預設值）
`0x01`	+ 1B 實際收到的長度	client 寫入長度錯誤 → NACK
`0x02`	+ 6B int16 X / Y / Z（little-endian）	每 500 ms 自動推送磁場原始值，可用於現場校正 / 即時觀測
`0x03`	+ status + counter + batt_level + batt_mode	每 1秒推送 Hook 當前狀態

* 廣播 manufacturer data 同時以同樣資料更新（每 1秒一次），未連線的被動掃描器也能讀到當下的 Hook 狀態與電量。

* 警報期間（`status = 1`）notify / 廣播都會帶著 `status = 1`，直到再次偵測到金屬為止。

2.5 設定下發（從 Belt / Web BLE Console）

* Belt（或維護用 Web BLE Console）可以透過 RX characteristic 寫入 17 byte `hook_downlink_command_struct_v2`：

* `system_cycle_time`（1B）：警報窗口秒數（1~30）。

* `flags`（1B）：旗標位元組（v3 起；v2 為已棄用的 `advertising_duration`，同位元組）。bit `0x80` = `IN_SAFE_ZONE`，Hook 收到後會 pin 住 `not_detected_counter` 為 0；其餘 bit 保留。`flags` 為 transient 不寫入 flash。詳見 [Belt-Hook 參數協定](../belt/parameter_protocol.md) §3。

* X / Y / Z 三軸（各 5B）：`sensing_type`、`sensitivity_high`、`sensitivity_low`。

* Hook 收到設定包後：

* 長度不符 → 回 `0x01` NACK + 實際長度，不寫入 flash。

* 長度正確 → 寫入 LittleFS，立即生效（不需重開機），並回 `0x00` ACK + 實際儲存值（含 firmware-side clamp 後的最終值）讓 client 確認。

2.6 AT 命令（USB / Serial）

維護或量產治具可透過 USB Serial 下達下列命令；都會立即套用並寫回 LittleFS：

命令	用途	範例
`AT+BATT_FACTOR=<float>`	電池 ADC 校正乘數（per-unit）	`AT+BATT_FACTOR=1.0339`
`AT+BATT_FACTOR=?`	查詢目前乘數	—
`AT+BATT_OFFSET=<float>`	電池 ADC 校正偏移（per-unit）	`AT+BATT_OFFSET=0.0000`
`AT+BATT_OFFSET=?`	查詢目前偏移	—
`AT+RESET_SETTING=1`	把所有設定（含警報窗口、三軸靈敏度、電池校正）還原成韌體預設值	—

> 量產備註：每顆 ESP32-C3 的 ADC chip-to-chip 變異約 3 %，每台 Hook 出廠都應接 4.08 V 標準電壓，量出當顆的 raw 讀值並寫入對應的 `AT+BATT_FACTOR`，才能讓上報的電量百分比準確。

---

3. 閒置省電（BLE-idle Duty Cycle）

* Hook 在「沒有 BLE client 連線」狀態下會啟動週期性省電：

* 持續廣播 4秒。

* 沒人連 → 進 deep sleep 1秒 → timer wake 後從 setup() 重啟。

* 整個循環約 80% 工作 / 20% 休眠，廣播仍然連續可被掃描到。

* 一旦有 client 連上，計時器立即歸零，連線期間完全不會進 deep sleep。

* 這個 duty-cycle 的代價：每次 deep sleep 重啟會把磁場警報的「未偵測累積時間」`last_metal_seen_ms` 重置（沒有 RTC 持久化）。這是有意接受的 trade-off — Hook 沒有 client 連線時通常是腰帶不在附近，沒有實際監控對象，不需要追蹤連續累積。

---

4. 充電模式（Charging Mode）

當開機時偵測到 `pCHARGING` 為 HIGH（充電器接著），Hook 直接進入充電模式，跳過正常的磁場感測與警報邏輯：

4.1 充電中（電量未滿）

* 一次完整循環：5秒 deep sleep → 喚醒短閃 1次（120 ms）→ BLE 廣播 2秒 → 重檢 `pCHARGING`。

* 每次喚醒會：

* 重新量測電池電壓（10個樣本平均，扣掉首樣 ADC 偏差），更新 `battery.level` / `battery.mode = POWER_CHARGING`。

* 更新廣播 manufacturer data 中的電量資訊。

* 短閃一次 LED 作為「充電中」心跳指示。

* 重檢 `pCHARGING`：

* 仍 HIGH → 再進 5秒 deep sleep。

* LOW（拔線）→ 進 1秒 deep sleep 後 reboot；給線上殘壓自然放電到 0 V，重開機讀到乾淨 LOW，setup() 走正常運作分支。

4.2 充電完成（電量已滿，電壓 ≥ 4.10 V）

* 切到 FULL awake loop：

* 兩顆 LED 透過 LEDC PWM 50% duty 點亮（5 kHz / 8-bit），看起來恆亮但耗電減半。

* BLE 持續廣播（不睡），場域管理者用手機 / 平板靠近時隨時可以看到「這支 Hook 已充飽」與最新電量。

* 每 5秒重新量電池電壓並更新 manufacturer data。

* 每秒 `digitalRead(pCHARGING)` 檢查充電器是否仍在。

* 遲滯切換（Hysteresis）：

* 進入 FULL：`v_corr ≥ 4.10 V`。

* 退出 FULL：`v_corr < 4.05 V`（chip + BLE 同時開所耗電會慢慢把電壓拉低；達到 4.05 V 時 deep sleep 1秒 reboot，下個循環走 §4.1 的低 duty 補電路徑）。

* 50 mV 緩衝避免 LED 在「恆亮」與「心跳閃爍」之間頻繁跳動。

* 拔線退出：1秒 deep sleep → reboot；setup() 讀到 `pCHARGING = LOW`，進入正常運作。

> 為什麼充電期間用 `digitalRead` 而不是 ADC 讀 `pCHARGING`？ 在 chip 持續喚醒的 FULL awake loop 中，重複的 ADC 取樣會把高阻抗的 charger STAT 線拉到 ~1.85 V（sample-hold 電容放電比充電 IC 補回快），造成誤判為「拔線」。`digitalRead` 是高阻抗輸入，不會干擾線上電壓。

---

5. 韌體無線更新（BLE OTA）

* Hook 透過 [h2zero/NimBLEOta](https://github.com/h2zero/esp-nimble-cpp) 提供 BLE OTA service，搭配 Device Information Service (DIS) 公開：

* `Manufacturer Name`：Siliqs

* `Model Number`：XDA003-H

* `Hardware Revision`：v0.2

* `Firmware Revision`：build script `scripts/inject_version.py` 注入的 `<sha>-<YYYYMMDD>` 字串

* 場域管理者可用支援 Web Bluetooth 的瀏覽器（Chrome / Edge）開啟維護 console，挑選 firmware 檔案後啟動 OTA。

* 更新期間 Hook 會暫停磁場 task 與 BLE notify（透過 `_ota_active` 旗標），把全部 NimBLE TX 頻寬讓給 OTA chunk 傳輸 — 否則 500 ms 一輪的磁場 notify 會把 host TX queue 塞滿，OTA 卡在第 0個 sector 完全傳不過。

* OTA 完成後：

* `onComplete` callback 會起一個 side task 延遲 2秒才呼叫 `ESP.restart()`，讓最後一個 sector 的 fwAck indicate 有機會傳出去並讓 client 確認，避免 client 在 99% 卡住 timeout。

* 重啟後 Hook 走 setup() 正常路徑，BLE 連線會自動恢復。

* 異常處理：

* `Disconnected` reason → 啟動 30秒 abort timer，給 client 重連的時間。

* `FlashError` reason → 立即 abort，並清掉 `_ota_active` 讓磁場 task 恢復，避免 Hook 卡在「無法工作但又無法 OTA」的 zombie 狀態。

* 一份完整韌體（約 750 KB）的更新時間視 client / 連線品質而定，bench 環境約 30~60秒。

> 注意：充電 cycle 的 5 秒喚醒視窗（§4.1）沒有掛載 OTA service，視窗太短（2 秒廣播）也不可能完成 OTA。要對 Hook 做 OTA 請在「正常運作」或「FULL awake loop」狀態下進行。

---

6. 電池狀態回報

* Hook 在每一份 BLE notify / 廣播 manufacturer data 中都會帶上下列電池資訊：

* `battery.level`（0~255）：對應 3.0~4.2 V 的線性映射。

* `battery.mode`：

* `POWER_NORMAL` — 正常運作中。

* `POWER_SAVING` — 電池過低（韌體判定後切換）。Phase 1 主要在 main loop 列印警告，尚未限縮 BLE / 磁感行為。

* `POWER_CHARGING` — 充電中（含 §4.1 與 §4.2 兩種子狀態）。

* Belt 收到後會把這些欄位放進對 LoRa Gateway 的上行包，後台據此追蹤 Hook 的電量與充電進度。